
QML Temelleri

Pratik Örneklerle Qt Quick

Bileşenler • Layout'lar • Modeller • Animasyonlar

Bu belge, Qt Quick / QML ile arayüz geliştirmede **en sık kullanılan** yapı taşlarını sıfırdan anlatır: görsel öğeler, özellik bağlama, konumlandırma, **yeniden kullanılabilir bileşenlere bölme**, sinyaller, durumlar, animasyonlar ve liste görünümleri. Her konu **çok sayıda kopyala-çalıştır örneğiyle** verilmiştir.

Kapsam: Item/Rectangle/Text/Image • property & binding • anchors • Row/Column/Grid & Layouts • **kendi bileşenini yazma** • property/signal API • sinyaller & handler • States & Transitions • animasyonlar & Behavior • ListModel/ListView/Repeater/GridView • Loader • Qt Quick Controls • en iyi pratikler

Hazırlanma Tarihi: 9 Temmuz 2026

Qt 6.x • QtQuick • QtQuick.Controls & Layouts

İçindekiler

1	QML Söz Dizimi: Temel Kavramlar	2
1.1	İlk QML Dosyası	2
1.2	Nesne, id ve Özellik	2
1.3	Property Binding: QML'in Kalbi	3
2	Temel Görsel Öğeler	3
2.1	Item — Görünmez Kapsayıcı	3
2.2	Rectangle — Dikdörtgen	3
2.3	Text — Metin	4
2.4	Image — Görsel	4
2.5	MouseArea — Fare/Dokunma Etkileşimi	4
3	Konumlandırma: Anchors ve Positioner'lar	5
3.1	Anchors — Bağlama	5
3.2	Row, Column, Grid — Positioner'lar	6
3.3	Layout'lar — Esnek ve Uyarlanabilir	6
4	Bileşenlere Bölme: Yeniden Kullanılabilir QML	7
4.1	İlk Özel Bileşen	7
4.2	property alias — İç Öğeyi Dışarı Açma	9
4.3	default property — İçerik Yuvası	9
4.4	inline Component ve Component Tipi	10
5	Sinyaller ve Handler'lar	10
5.1	Kendi Sinyalini Tanımlama ve Yayma	10
5.2	Connections ile Dışarıdan Bağlanma	10
5.3	Property Değişim Sinyalleri	11
6	Durumlar (States) ve Geçişler (Transitions)	11
6.1	States ile Görünüm Değiştirme	11
6.2	Transitions ile Yumuşak Geçiş	11
7	Animasyonlar	12
7.1	Behavior — Otomatik Yumuşatma	12
7.2	Sıralı ve Paralel Animasyonlar	12
8	Modeller ve Görünümler: Liste Göstermek	13
8.1	Repeater — En Basit Tekrarlama	13
8.2	ListModel + ListView — Kaydırılabilir Liste	13
8.3	ListModel'i Dinamik Değiştirme	14
8.4	GridView — Izgara Görünüm	14
9	Loader ve Dinamik İçerik	15
10	Qt Quick Controls: Hazır Bileşenler	16
11	En İyi Pratikler ve Sık Hatalar	17
11.1	Yapılması Gerekenler	17
11.2	Kaçınılması Gerekenler	17
11.3	Hızlı Referans	18

1. QML Söz Dizimi: Temel Kavramlar

QML, kullanıcı arayüzünü *ne olduğunu* tanımlayarak (deklaratif) yazdığımız bir dildir. Bir QML dosyası, iç içe geçmiş **nesnelerden** oluşan bir ağaçtır. Her nesnenin *özellikleri* (property) vardır ve bu özellikler birbirine *bağlanabilir* (binding).

1.1 İlk QML Dosyası

```

1 import QtQuick                // Temel QML tipleri (Item, Rectangle, Text...)
2 import QtQuick.Window        // Window tipi
3
4 Window {
5     width: 400
6     height: 300
7     visible: true
8     title: "İlk QML Uygulamam"
9
10    Rectangle {
11        anchors.centerIn: parent // Pencerenin ortasına yerleştir
12        width: 200
13        height: 100
14        color: "steelblue"
15        radius: 12                // Yuvarlatılmış köşeler
16
17        Text {
18            anchors.centerIn: parent
19            text: "Merhaba QML"
20            color: "white"
21            font.pixelSize: 20
22        }
23    }
24 }
```

1.2 Nesne, id ve Özellik

Her QML nesnesi `TipAdi { ... }` biçiminde yazılır. İçine özellik atamaları ve alt nesneler koyarsınız.

Öge

Açıklama

id	Nesneye benzersiz kimlik; başka nesnelerden erişmek için. Örn: <code>id: box</code>
property	Özellik: nesnenin verisi. <code>width</code> , <code>color</code> , <code>text</code> ...
parent	Bir nesnenin görsel ebeveyni; <code>anchors</code> ve boyutlandırmada kullanılır.
{ }	Nesne kapsamı; içine alt nesneler ve özellikler yazılır.

İpucu

`id` bir string değildir, tırnak almaz: `id: baslik`. Aynı dosyada benzersiz olmalı, küçük harfle başlamalıdır. Bir nesneye başka yerden erişmenin tek yolu `id`'sidir: `baslik.text = "Yeni"`.

1.3 Property Binding: QML'in Kalbi

Bir özelliği sabit değil, *bir ifadeye* bağlayabilirsiniz. İfadedeki değerler değişince özellik **otomatik güncellenir**.

```
1 Rectangle {
2     id: box
3     width: 200
4     // height artık width'e bağlı; width değişince height da değişir
5     height: width / 2
6     color: width > 150 ? "green" : "red" // Kosula bağlı renk
7 }
```

i Bilgi

Binding tek yönlü ve canlıdır. `height: width / 2` yazdığınızda, "şu an width'in yarısını al" değil, "her zaman width'in yarısı ol" dersiniz. width 300 olursa height anında 150 olur. Elle güncelleme kodu yazmazsınız — bu, QML'i güçlü kılan özelliktir.

2. Temel Görsel Öğeler

QtQuick'te en sık kullanacağınız birkaç temel tip vardır. Bunları öğrenmek arayüzlerin %80'ini kapsar.

2.1 Item — Görünmez Kapsayıcı

Item tüm görsel öğelerin taban tipidir; kendi başına bir şey çizmez ama gruplamak, konumlandırmak ve boyutlandırmak için kullanılır.

```
1 Item {
2     width: 300; height: 200
3     // İçindeki öğeleri gruplar; kendisi görünmez
4     Rectangle { anchors.fill: parent; color: "lightgray" }
5 }
```

2.2 Rectangle — Dikdörtgen

```
1 Rectangle {
2     width: 120; height: 80
3     color: "tomato"
4     radius: 10 // Kose yuvarlaklığı
5     border.color: "darkred" // Kenarlık rengi
6     border.width: 2 // Kenarlık kalınlığı
7     opacity: 0.9 // Saydamlık (0=görünmez, 1=opak)
8
9     // Renk geçisi (gradient)
10    gradient: Gradient {
11        GradientStop { position: 0.0; color: "orange" }
12        GradientStop { position: 1.0; color: "red" }
13    }
14 }
```

2.3 Text — Metin

```

1 Text {
2   text: "Uzun bir metin bu satıra sığmayabilir"
3   color: "#333333"
4   font.pixelSize: 18
5   font.bold: true
6   font.family: "Arial"
7   width: 150 // Sabit genişlik
8   wrapMode: Text.WordWrap // Kelime bazlı alt satıra geç
9   elide: Text.ElideRight // Sığmazsa sonuna "..." koy
10  horizontalAlignment: Text.AlignHCenter
11 }

```

2.4 Image — Görsel

```

1 Image {
2   source: "qrc:/images/logo.png" // veya "https://..." / dosya yolu
3   width: 100; height: 100
4   fillMode: Image.PreserveAspectFit // Oranı koru, sığdır
5   asynchronous: true // Arka planda yükle (UI donmasın)
6 }

```

fillMode	Davranış
Image.Stretch	Görseli çerçeveye uzat (oran bozulur)
Image.PreserveAspectFit	Oranı koruyarak sığdır (boşluk kalabilir)
Image.PreserveAspectCrop	Oranı koruyarak doldur (taşan kırpılır)
Image.Tile	Döşe (tekrarla)

2.5 MouseArea — Fare/Dokunma Etkileşimi

Görsel öğeler kendiliğinden tıklanabilir değildir; üzerlerine görünmez bir **MouseArea** koyarsınız.

```

1 Rectangle {
2   id: button
3   width: 120; height: 40
4   color: mouse.pressed ? "darkblue" : "steelblue" // Basiliyken koyulasir
5   radius: 6
6
7   Text { anchors.centerIn: parent; text: "Tikla"; color: "white" }
8
9   MouseArea {
10    id: mouse
11    anchors.fill: parent // Tüm dikdörtgeni kapla
12    hoverEnabled: true // Fare üzerine gelince de algıla
13    onClicked: console.log("Tiklandi!")
14    onDoubleClicked: console.log("Cift tiklandi")
15    onPressed: console.log("Basildi")
16    onEntered: button.scale = 1.1 // Fare geldi
17    onExited: button.scale = 1.0 // Fare cikti
18 }

```

```
19 }
```

Modern alternatif: Handler'lar

Qt 6'da **MouseArea** yerine daha esnek **Input Handler**'lar önerilir: **TapHandler** (tıklama), **DragHandler** (sürükleme), **HoverHandler** (fare üzeri), **PinchHandler** (kıştırma). Bunlar birden fazla aynı anda çalışabilir ve dokunmatik ekranlarla daha uyumludur.

```
1 Rectangle {
2     width: 100; height: 100; color: "coral"
3     TapHandler { onTapped: console.log("TapHandler ile tiklandi") }
4     DragHandler { } // Tek satirli surukle-birak!
5 }
```

3. Konumlandırma: Anchors ve Positioner'lar

QML'de öğeleri yerleştirmenin üç yolu vardır: sabit x/y, **anchors** (bağlama), **positioner**'lar (Row/Column/Grid) ve **Layout**'lar.

3.1 Anchors — Bağlama

Bir öğenin kenarlarını başka bir öğenin kenarlarına "yapıştırırız".

```
1 Rectangle {
2     id: container
3     width: 300; height: 200; color: "#eee"
4
5     Rectangle {
6         width: 80; height: 80; color: "teal"
7         anchors.top: parent.top // Ust kenari ebeveynin ustune
8         anchors.left: parent.left // Sol kenari ebeveynin soluna
9         anchors.margins: 10 // 10px bosluk birak
10    }
11    Rectangle {
12        width: 80; height: 80; color: "orange"
13        anchors.centerIn: parent // Tam ortalama
14    }
15    Rectangle {
16        height: 40; color: "purple"
17        anchors.left: parent.left
18        anchors.right: parent.right // Sol+sag -> genislik otomatik
19        anchors.bottom: parent.bottom
20    }
21 }
```

Anchor	Açıklama
<code>anchors.fill: parent</code>	Ebeveyni tamamen kapla
<code>anchors.centerIn: parent</code>	Yatay+dikey ortalama
<code>anchors.top / bottom / left / right</code>	Tek kenar bağlama
<code>anchors.horizontalCenter</code>	Yatay merkez hizalama
<code>anchors.margins / topMargin ...</code>	Bağlanan kenardan boşluk

⚠ Dikkat

Bir öğeyi hem `anchors` ile hem de `x/y` ile konumlandırmayın — çakışır. Ayrıca `anchors` yalnızca **ebeveyn** veya **kardeş** öğelere bağlanabilir, rastgele bir öğeye değil.

3.2 Row, Column, Grid — Positioner'lar

Öğeleri otomatik olarak yatay, dikey veya ızgara biçiminde dizerler.

```

1 Column {                               // Alt alta dizer
2     spacing: 10                         // Öğeler arası boşluk
3     Rectangle { width: 100; height: 30; color: "red" }
4     Rectangle { width: 100; height: 30; color: "green" }
5     Rectangle { width: 100; height: 30; color: "blue" }
6 }
7
8 Row {                                  // Yan yana dizer
9     spacing: 8
10    Repeater {                          // 5 tane üret
11        model: 5
12        Rectangle { width: 40; height: 40; color: "steelblue"; radius: 4 }
13    }
14 }
15
16 Grid {                               // Izgara: 3 sütun
17     columns: 3
18     spacing: 6
19     Repeater {
20         model: 9
21         Rectangle { width: 50; height: 50; color: "orange" }
22     }
23 }
```

3.3 Layout'lar — Esnek ve Uyarlanabilir

`RowLayout`, `ColumnLayout`, `GridLayout`; öğelerin pencere boyutuna göre *esnemesini* sağlar. Bunun için `QtQuick.Layouts` import edilir ve öğeler `Layout.*` ekli özellikleriyle yapılandırılır.

```

1 import QtQuick
2 import QtQuick.Layouts
3
4 RowLayout {
5     anchors.fill: parent
6     spacing: 10
7 }
```

```

8   Rectangle {
9       color: "salmon"
10      Layout.preferredWidth: 100
11      Layout.fillHeight: true      // Dikeyde tum alani doldur
12  }
13  Rectangle {
14      color: "khaki"
15      Layout.fillWidth: true      // Kalan yatay alani KAP
16      Layout.fillHeight: true
17  }
18  Rectangle {
19      color: "lightgreen"
20      Layout.preferredWidth: 60
21      Layout.fillHeight: true
22  }
23  }

```

Layout özelliği	Açıklama
Layout.fillWidth / fillHeight	Kalan alanı doldur (esne)
Layout.preferredWidth / Height	Tercih edilen boyut
Layout.minimumWidth / maximumWidth	Alt/üst boyut sınırı
Layout.alignment	Hücre içi hizalama (Qt.AlignCenter...)
Layout.columnSpan / rowSpan	GridLayout'ta hücre yayılımı

💡 Positioner mı, Layout mu?

Row/Column basit, sabit boyutlu dizilimler için hafif ve hızlıdır. RowLayout/ColumnLayout ise öğelerin **pencere boyutuna göre esnemesi** gerektiğinde (responsive arayüz) kullanılır. Layout.fillWidth gibi özellikler sadece Layout'lar içinde çalışır, Row/Column içinde çalışmaz.

4. Bileşenlere Bölme: Yeniden Kullanılabilir QML

QML'in en güçlü tarafı: **her .qml dosyası aynı zamanda bir tiptir**. MyButton.qml adında bir dosya oluşturursanız, başka QML dosyalarında MyButton { } yazarak onu bir öğe gibi kullanabilirsiniz. Böylece tekrar eden arayüz parçalarını bileşenlere bölersiniz.

4.1 İlk Özel Bileşen

Diyelim aynı stildeki düğmeyi defalarca kullanacaksınız. Aynı bir dosyaya taşıyın:

🔗 StyledButton.qml — kendi bileşenimiz

```

1  import QtQuick
2
3  Rectangle {                                // Kok oge = bilesenin "govdesi"
4      id: root
5      width: 140; height: 44
6      radius: 8

```



```

7   color: area.pressed ? Qt.darker(baseColor, 1.2)
8                               : (area.containsMouse ? Qt.lighter(baseColor, 1.1)
9                               : baseColor)
10
11  // ---- DISARIYA ACILAN API ----
12  property string text: "Dugme"      // Ozellestirilebilir etiket
13  property color baseColor: "#2E6DA4"
14  signal clicked()                  // Disariya bildirilen sinyal
15
16  Text {
17      anchors.centerIn: parent
18      text: root.text                // Disaridan gelen metin
19      color: "white"
20      font.pixelSize: 16
21  }
22
23  MouseArea {
24      id: area
25      anchors.fill: parent
26      hoverEnabled: true
27      onClicked: root.clicked()      // Kendi sinyalimizi yay
28  }
29  }

```

Artık başka dosyada kullanabiliriz (aynı klasördeyse import bile gerekmez):

```

1  import QtQuick
2  import QtQuick.Window
3
4  Window {
5      width: 300; height: 240; visible: true
6
7      Column {
8          anchors.centerIn: parent
9          spacing: 12
10
11         StyledButton {
12             text: "Kaydet"
13             baseColor: "#2E7D32"
14             onClicked: console.log("Kaydedildi") // Sinyali dinle
15         }
16         StyledButton {
17             text: "Sil"
18             baseColor: "#C0392B"
19             onClicked: console.log("Silindi")
20         }
21         StyledButton { text: "Iptal" } // Varsayilan mavi
22     }
23 }

```

Bilgi

Bir bileşenin **genel API'si** üç şeyden oluşur: **(1)** dışarıdan ayarlanabilen property'ler (girdi), **(2)** dışarıya olay bildiren signal'ler (çıkış), **(3)** çağrılabilen function'lar. İç yapı (MouseArea, Text vs.) gizli kalır — kullanıcı sadece bu API'yi görür. Bu, kapsülleme (encapsulation) demektir.

4.2 property alias — İç Öğeyi Dışarı Açma

Bazen iç öğenin bir özelliğine dışarıdan doğrudan erişim vermek istersiniz. `alias` bunu sağlar:

```
1 // LabeledField.qml
2 import QtQuick
3 import QtQuick.Controls
4
5 Row {
6     spacing: 8
7     property alias label: labelText.text // Ic Text'in text'ini dışarı aç
8     property alias value: input.text // Ic TextField'in text'ini aç
9
10    Text { id: labelText; text: "Ad:" }
11    TextField { id: input; placeholderText: "..." }
12 }
```

```
1 LabeledField {
2     label: "E-posta:"
3     value: "ornek@mail.com"
4     onValueChanged: console.log("Değer:", value) // alias otomatik sinyal üretir
5 }
```

property vs alias

`property string x: "..."` yeni bir değer saklar. `property alias x: oge.y` ise mevcut bir öğenin özelliğine *takma ad* verir — okuma ve yazma doğrudan o öğeye gider. Alias, iç bir bileşenin özelliğini/nesnesini API'ye dahil etmenin temiz yoludur.

4.3 default property — İçerik Yuvası

Kendi bileşeninize, tıpkı `Column` gibi, doğrudan çocuk öğe alabilme yeteneği verebilirsiniz:

```
1 // Card.qml -- icine konan her sey "content" alanına gider
2 import QtQuick
3
4 Rectangle {
5     id: root
6     radius: 10; color: "white"; border.color: "#ddd"
7     implicitWidth: content.implicitWidth + 24
8     implicitHeight: content.implicitHeight + 24
9
10    default property alias data: content.data // <-- Sihir burada
11
12    Column {
13        id: content
14        anchors.centerIn: parent
15        spacing: 8
16    }
17 }
```

```
1 Card {
2     // Bunlar dogrudan Card'in icine (content Column'a) gider:
3     Text { text: "Baslik"; font.bold: true }
4     Text { text: "Aciklama metni burada" }
5 }
```

4.4 inline Component ve Component Tipi

Küçük, sadece o dosyada kullanılacak bileşenler için ayrı dosya açmadan component anahtar kelimesiyle satır içi tip tanımlayabilirsiniz:

```

1 Item {
2     // Dosya içi bileşen tanımı (Qt 6)
3     component Badge: Rectangle {
4         width: 60; height: 24; radius: 12; color: "purple"
5         property alias text: t.text
6         Text { id: t; anchors.centerIn: parent; color: "white" }
7     }
8
9     Row {
10        spacing: 6
11        Badge { text: "Yeni" }
12        Badge { text: "Pro"; color: "orange" }
13    }
14 }
```

5. Sinyaller ve Handler'lar

Sinyaller, "bir şey oldu" bilgisini iletir. Yerleşik öğelerin hazır sinyalleri (onClicked, onPressed...) vardır; kendi sinyallerinizi de tanımlayabilirsiniz.

5.1 Kendi Sinyalini Tanımlama ve Yayma

```

1 Item {
2     id: box
3
4     // Parametrelili sinyal tanımı (Qt 6 tarzı)
5     signal itemSelected(int index, string name)
6
7     function select(i, name) {
8         box.itemSelected(i, name)    // Sinyali yay
9     }
10
11     // Aynı Item içinde handler:
12     onItemSelected: (index, name) => {
13         console.log("Secilen:", index, name)
14     }
15 }
```

5.2 Connections ile Dışarıdan Bağlanma

Bir öğenin sinyaline, tanımından başka bir yerde bağlanmak için:

```

1 Timer {
2     id: timer
3     interval: 1000; running: true; repeat: true
4 }
5
6 Connections {
7     target: timer
8     function onTriggered() {    // Timer'in "triggered" sinyali
```

```

9     console.log("Bir saniye gecti")
10 }
11 }

```

5.3 Property Değişim Sinyalleri

Her özellik otomatik olarak bir `on<Özellik>Changed` handler'ı sağlar:

```

1 Rectangle {
2     width: 100; height: 100
3     // width degistiginde otomatik cagrilir
4     onWidthChanged: console.log("Yeni genislik:", width)
5     color: "gray"
6     onColorChanged: console.log("Renk degisti")
7 }

```

6. Durumlar (States) ve Geçişler (Transitions)

Bir öğenin farklı görünümlerini (açık/kapalı, seçili/seçilmemiş) **state** olarak tanımlar, aralarındaki *yumuşak geçişi* **transition** ile sağlarsınız.

6.1 States ile Görünüm Değiştirme

```

1 Rectangle {
2     id: panel
3     width: 200; height: 60
4     color: "steelblue"
5
6     // Baslangicta state yok (varsayilan); tanimli durumlar:
7     states: [
8         State {
9             name: "expanded"
10            PropertyChanges { target: panel; height: 200; color: "seagreen" }
11        },
12        State {
13            name: "hidden"
14            PropertyChanges { target: panel; height: 0; opacity: 0 }
15        }
16    ]
17
18    MouseArea {
19        anchors.fill: parent
20        // Tikla -> durumu degistir (bosluk = varsayilana don)
21        onClicked: panel.state = (panel.state === "expanded")
22                        ? "" : "expanded"
23    }
24 }

```

6.2 Transitions ile Yumuşak Geçiş

State değişimi ani olur; **Transition** eklerseniz animasyonlu olur:

```

1 Rectangle {
2     id: box

```

```

3   width: 100; height: 100; color: "coral"
4   states: State {
5       name: "large"
6       PropertyChanges { target: box; width: 200; height: 200 }
7   }
8   transitions: Transition {
9       // Tüm sayısal özellikleri 300ms'de yumusakca degistir
10      NumberAnimation {
11          properties: "width,height"
12          duration: 300
13          easing.type: Easing.InOutQuad
14      }
15  }
16  MouseArea { anchors.fill: parent
17              onClicked: box.state = box.state === "large" ? "" : "large" }
18  }

```

7. Animasyonlar

QML animasyonlar için zengin bir sistem sunar. En sık kullanılanlar:

7.1 Behavior — Otomatik Yumuşatma

Bir özellik *her ne zaman* değişirse değişsin animasyonlu olsun istiyorsanız **Behavior** en pratik yoldur:

```

1  Rectangle {
2      id: ball
3      width: 60; height: 60; radius: 30; color: "tomato"
4      x: 0
5
6      // x her degistiginde 400ms'de yumusakca kaysin
7      Behavior on x {
8          NumberAnimation { duration: 400; easing.type: Easing.OutBounce }
9      }
10
11     MouseArea {
12         anchors.fill: parent
13         onClicked: ball.x = ball.x === 0 ? 200 : 0 // x'i degistir, animasyon otomatik
14     }
15 }

```

7.2 Sıralı ve Paralel Animasyonlar

```

1  Rectangle {
2      id: square
3      width: 80; height: 80; color: "mediumpurple"
4
5      // Tiklanınca calisacak animasyon zinciri
6      SequentialAnimation {
7          id: anim
8          NumberAnimation { target: square; property: "rotation"
9                          to: 360; duration: 500 }
10         ParallelAnimation { // Ayni anda:
11             NumberAnimation { target: square; property: "scale"
12                             to: 1.5; duration: 300 }

```

```

13         ColorAnimation { target: square; property: "color"
14                           to: "gold"; duration: 300 }
15     }
16     NumberAnimation { target: square; property: "scale"
17                       to: 1.0; duration: 200 }
18 }
19 MouseArea { anchors.fill: parent; onClicked: anim.start() }
20 }

```

Animasyon tipi	Kullanım
NumberAnimation	Sayısal özellikler (x, width, opacity...)
ColorAnimation	Renk geçişleri
RotationAnimation	Dönüş (en kısa yol seçebilir)
SequentialAnimation	İçindekileri sırayla çalıştırır
ParallelAnimation	İçindekileri aynı anda çalıştırır
Behavior on X	X değişince otomatik animasyon
PropertyAnimation	Genel; her tip özelliğe uygulanır

💡 Easing — yumuşama eğrileri

`easing.type` animasyonun "hissini" belirler: `Easing.Linear` (sabit hız), `Easing.InOutQuad` (yavaş başla-hızlan-yavaşla), `Easing.OutBounce` (zıplayarak dur), `Easing.OutElastic` (yaylanarak). Doğal his için genelde `InOutQuad` veya `OutCubic` iyi başlangıçtır.

8. Modeller ve Görünümler: Liste Göstermek

Tekrar eden veriyi (liste, ızgara) göstermenin yolu **model-view** ayrımıdır: *model* veriyi tutar, *delegate* her öğenin nasıl görüneceğini tanımlar, *view* (`ListView`/`GridView`) ikisini birleştirir.

8.1 Repeater — En Basit Tekrarlama

```

1 Column {
2     spacing: 4
3     Repeater {
4         model: ["Elma", "Armut", "Muz"] // Dizi model
5         delegate: Text {
6             required property string modelData // Dizi elemanı
7             required property int index
8             text: (index + 1) + ". " + modelData // "1. Elma"
9         }
10    }
11 }

```

8.2 ListModel + ListView — Kaydırılabilir Liste

```

1 import QtQuick
2
3 ListView {
4     width: 250; height: 300
5     spacing: 6
6     clip: true // Tasan icerigi kirp
7
8     model: ListModel {
9         ListElement { name: "Ali"; age: 25 }
10        ListElement { name: "Ayse"; age: 30 }
11        ListElement { name: "Veli"; age: 28 }
12    }
13
14    delegate: Rectangle {
15        // Model rolleri delegate icinde dogrudan erisilir
16        required property string name
17        required property int age
18        width: ListView.view.width
19        height: 50
20        color: "#f0f4f8"
21        radius: 6
22
23        Row {
24            anchors.verticalCenter: parent.verticalCenter
25            anchors.left: parent.left; anchors.leftMargin: 12
26            spacing: 12
27            Text { text: name; font.bold: true }
28            Text { text: age + " yas"; color: "gray" }
29        }
30    }
31 }

```

Dikkat

Delegate içinde model rollerine erişmek için Qt 6'da **required property** kullanımı **önerilir** (required property string ad). Eski kodda bu roller sihirli biçimde kapsamdaydı ama isim çakışması ve karışıklık yaratıyordu. required ile hangi rollerin kullanıldığı açık ve güvenli olur.

8.3 ListModel'i Dinamik Değiştirme

```

1 ListModel { id: list }
2
3 // Ekle / sil / guncelle
4 function add(name) { list.append({ name: name, age: 0 }) }
5 function remove(i) { list.remove(i) }
6 function clear() { list.clear() }
7 function update(i, newName) { list.setProperty(i, "name", newName) }

```

8.4 GridView — Izgara Görünüm

```

1 GridView {
2     width: 320; height: 300
3     cellWidth: 100; cellHeight: 100 // Her hucrenin boyutu
4     model: 12

```

```

5   delegate: Rectangle {
6       required property int index
7       width: 90; height: 90
8       color: Qt.hsla(index / 12, 0.6, 0.6, 1) // Renk carkı
9       radius: 8
10      Text { anchors.centerIn: parent; text: index; color: "white" }
11  }
12 }

```

Özellik

Açıklama

model	Veri kaynağı (sayı, dizi, ListModel, C++ modeli)
delegate	Her öğenin görsel şablonu
clip: true	Görünüm dışına taşan öğeleri gizle
spacing	Öğeler arası boşluk (ListView)
header / footer	Liste başı/sonu öğeleri
ScrollBar.vertical	Kaydırma çubuğu ekleme

9. Loader ve Dinamik İçerik

Loader, bir bileşeni *gerektiğinde* yükler — sekmeli arayüzlerde, koşullu içerikte ve belleği verimli kullanmada idealdir.

```

1  Item {
2      Loader {
3          id: loader
4          anchors.fill: parent
5          // Kaynagi degistirerek icerigi degistir
6          source: currentPage === "settings" ? "SettingsPage.qml" : "HomePage.qml"
7      }
8
9      // veya satir ici bilesen yukle:
10     Loader {
11         sourceComponent: isOpen ? detailComponent : null
12     }
13     Component {
14         id: detailComponent
15         Rectangle { color: "lightyellow"; width: 100; height: 100 }
16     }
17 }

```

İpucu

`Loader.active: false` yaparak yüklenen içeriği bellekten kaldırabilirsiniz. Ağır sayfaları yalnızca görünür olduklarında yüklemek (lazy loading) uygulama başlangıcını hızlandırır. `asynchronous: true` ile yükleme arka planda olur.

10. Qt Quick Controls: Hazır Bileşenler

QtQuick.Controls; düğme, metin kutusu, kaydırıcı gibi hazır, stillenebilir arayüz bileşenleri sunar. Sıfırdan yazmak yerine bunları kullanın.

```

1 import QtQuick
2 import QtQuick.Controls
3 import QtQuick.Layouts
4
5 ColumnLayout {
6     anchors.centerIn: parent
7     spacing: 10
8
9     Button {
10         text: "Tikla"
11         onClicked: statusLabel.text = "Butona basildi"
12     }
13
14     TextField {
15         placeholderText: "Adinizi girin"
16         onTextChanged: console.log("Metin:", text)
17     }
18
19     Slider {
20         from: 0; to: 100; value: 50
21         onValueChanged: console.log("Deger:", value.toFixed(0))
22     }
23
24     CheckBox {
25         text: "Sartlari kabul ediyorum"
26         onCheckedChanged: console.log("Onay:", checked)
27     }
28
29     ComboBox {
30         model: ["Kucuk", "Orta", "Buyuk"]
31         onCurrentTextChanged: console.log("Secim:", currentText)
32     }
33
34     Label { id: statusLabel; text: "Hazir" }
35 }

```

Control	Kullanım
Button	Tıklanabilir düğme (onClicked)
TextField / TextArea	Tek/çok satır metin girişi
Slider / RangeSlider	Kaydırıcı ile değer seçimi
CheckBox / Switch	Aç/kapa seçimi
RadioButton	Gruptan tek seçim
ComboBox	Açılır liste
Dialog / Popup	İletişim kutusu / açılır pencere
TabBar / StackView	Sekme ve sayfa yığını navigasyonu

Stil seçimi

Qt Quick Controls farklı stillerle gelir: Basic, Material, Universal, Fusion, macOS, Windows. Uygulama genelinde stili `main.cpp`'de `QQuickStyle::setStyle("Material")` ile veya `qtquickcontrols2.conf` dosyasıyla ayarlarsınız. Material stil, renk ve tema özelleştirmesi için en esnek olanıdır.

11. En İyi Pratikler ve Sık Hatalar

11.1 Yapılması Gerekenler

- **Bileşenlere böl:** 100+ satırlık QML dosyalarını mantıklı, yeniden kullanılabilir bileşenlere ayırın. Her bileşen tek bir işi yapsın.
- **Net API tasarla:** Bileşenin dışarıya açtığı property ve signal'leri bilinçli seçin; iç yapıyı gizleyin.
- **Binding kullan, elle güncelleme yapma:** `width: parent.width / 2` yaz; `onParentWidthChanged` ile elle atama yapma.
- **Layout'larla responsive tasarla:** Sabit piksel yerine anchors ve Layout kullan; farklı ekranlara uyum sağlasın.
- **required property kullan:** Delegate'lerde model rollerini açıkça belirt.
- **Ağır iş QML'de yapma:** Karmaşık hesaplama/döngüleri C++'a taşı; QML sadece sunum olsun.

11.2 Kaçınılması Gerekenler

Hata	Neden kaçınmalı
Binding'i JS ile ezmek	<code>width = 100</code> atarsanız binding kalıcı olarak kırılır . İfade tabanlı kalın.
anchors + x/y karıştırmak	Çakışır, öngörülemez sonuç. Birini seçin.
Dev tek dosya	Bakımı imkânsızlaşır. Bileşenlere bölün.
Delegate'te ağır iş	Kaydırma takılır. Delegate hafif olmalı; <code>clip</code> ve <code>asynchronous</code> kullanın.
id yerine sabit yol	<code>parent.parent.parent...</code> kırılabilir. <code>id</code> ile referans verin.
console.log unutmak	Çalışmayan binding'i sessizce kaçırsınız. Şüphede logla.

11.3 Hızlı Referans

Öge	Amaç
Item / Rectangle / Text / Image	Temel görsel yapı taşları
MouseArea / TapHandler	Kullanıcı etkileşimi
anchors	Öğeyi ebeveyn/kardeşe bağlama
Row / Column / Grid	Otomatik dizilim
RowLayout / ColumnLayout	Esneyen (responsive) yerleşim
property / alias / signal	Bileşen API'si
States / Transitions	Görünüm durumları ve geçişleri
Behavior / NumberAnimation	Animasyon
ListView / GridView / Repeater	Liste ve ızgara görünümleri
Loader	Dinamik/tembel yükleme
Qt Quick Controls	Hazır arayüz bileşenleri

Özet: QML deklaratifdir — *ne* istediğinizi yazarsınız, *nasıl* güncelleneceğini binding halleder. Arayüzü küçük, net API'li bileşenlere bölün; yerleşimi `anchors` ve `Layout`'larla responsive yapın; tekrarlı veriyi model-view ile gösterin; hareketi `States/Behavior` ile canlandırın. İnce bir QML katmanı + ayrı bileşenler = bakımı kolay, ölçeklenebilir arayüz.